

CrowdStrike Falcon Container Sensor — AWS Batch + ECS

Fargate Deployment Guide

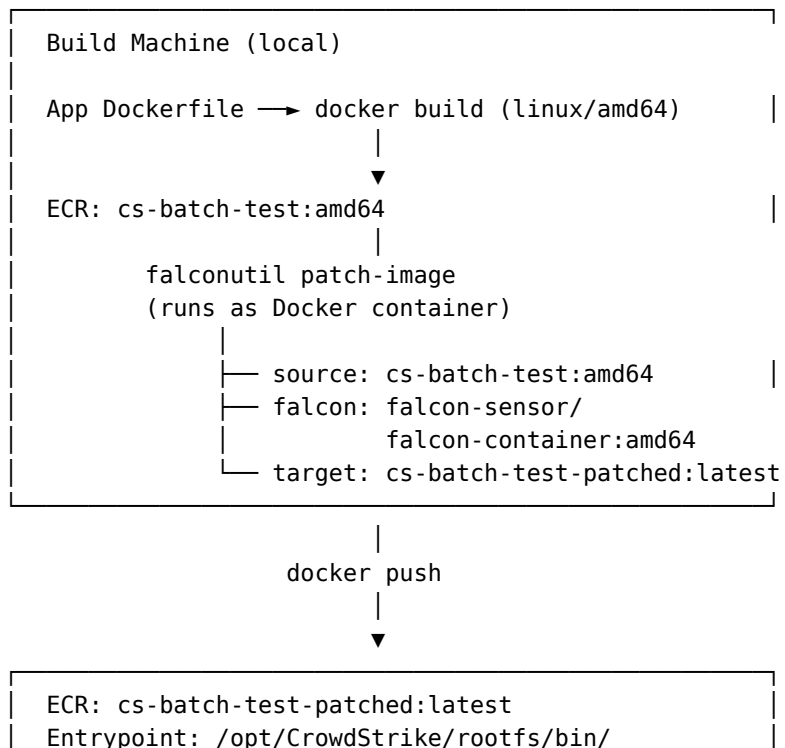
Validated: 2026-07-09 **Account:** 597047870845 (us-east-1) **Falcon Sensor Version:** 7.39.0-7802

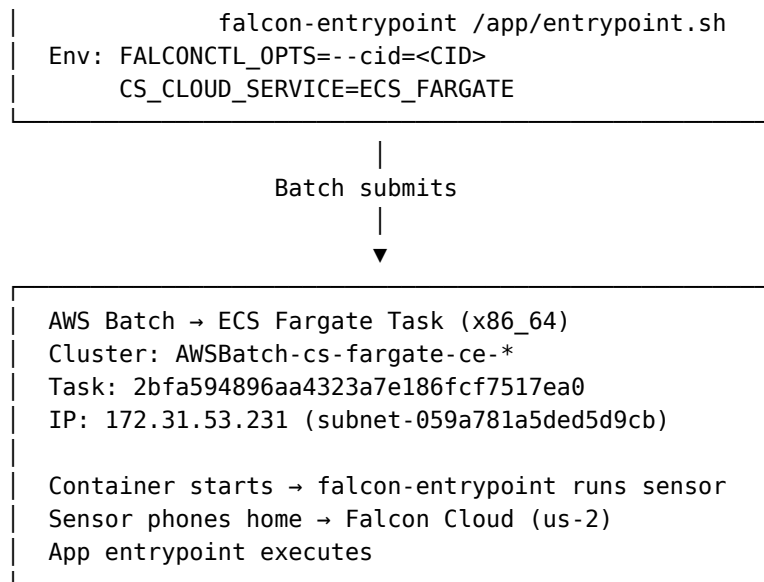
Approach: falconutil image patching (ECS_FARGATE mode)

Table of Contents

1. Architecture Overview
 2. Prerequisites Validation
 3. Build and Push the Application Image
 4. Pull and Push the Falcon Sensor Image
 5. Patch the Application Image with falconutil
 6. Inspect the Patched Image
 7. AWS Batch Infrastructure Setup
 8. Job Definition Details
 9. Submit and Monitor the Batch Job
 10. CloudWatch Log Verification
 11. ECS Task Verification
 12. Falcon Console Verification
 13. Troubleshooting Reference
-

Architecture Overview





Prerequisites Validation

1. AWS Identity

```
$ aws sts get-caller-identity
{
  "UserId": "AIDAYWAWTQV6ROYESN524",
  "Account": "597047870845",
  "Arn": "arn:aws:iam::597047870845:user/ultimate_demo"
}
```

What to check: Confirm the account ID matches your target environment. The IAM user/role needs permissions for ECR (push/pull), Batch (create compute env, job queue, job definition, submit job), ECS (describe tasks), CloudWatch Logs (create log group, put events).

2. Docker Runtime

```
$ docker info --format '{{.ServerVersion}}'
27.3.1
```

What to check: Docker must be running and able to build multi-architecture images. On Apple Silicon (arm64) Macs, you **must** explicitly build and push linux/amd64 images for ECS Fargate — Fargate only supports x86_64 unless you specifically select arm64.

3. ECR Authentication

```
LOGIN_PASS=$(aws ecr get-login-password --region us-east-1)
echo "$LOGIN_PASS" | docker login --username AWS --password-stdin \
  597047870845.dkr.ecr.us-east-1.amazonaws.com
# Login Succeeded
```

Important — macOS Keychain issue: The standard Docker config.json on macOS uses "credsStore": "osxkeychain". When falconutil runs inside a Linux container and mounts your config.json, it cannot read credentials from macOS Keychain because the keychain binary doesn't exist inside the container. The fix is documented in Phase 4.

Build and Push the Application Image

Dockerfile

```
FROM alpine:3.19
RUN apk add --no-cache bash curl
WORKDIR /app
COPY entrypoint.sh .
RUN chmod +x entrypoint.sh
ENTRYPOINT ["/app/entrypoint.sh"]
```

entrypoint.sh

```
#!/bin/bash
set -euo pipefail
echo "=== CS Batch Test Job Starting ==="
echo "Hostname: $(hostname)"
echo "Date: $(date -u)"
COUNTER=0
while true; do
    COUNTER=$((COUNTER + 1))
    echo "[$(date -u +%H:%M:%S)] Iteration $COUNTER – workload running"
    sleep 30
done
```

Build for linux/amd64

```
# IMPORTANT: Always specify --platform linux/amd64 when building on Apple Silicon
docker buildx build --platform linux/amd64 -t cs-batch-test:amd64 --load .
```

What to verify: The build output should show it fetching x86_64 Alpine packages, not aarch64:

```
#6 0.227 fetch https://dl-cdn.alpinelinux.org/alpine/v3.19/main/x86_64/APKINDEX.
tar.gz
```

If you see aarch64 here, you forgot --platform linux/amd64.

Create ECR Repo and Push

```
aws ecr create-repository --repository-name cs-batch-test --region us-east-1
```

```
docker tag cs-batch-test:amd64 \
    597047870845.dkr.ecr.us-east-1.amazonaws.com/cs-batch-test:amd64
```

```
docker push 597047870845.dkr.ecr.us-east-1.amazonaws.com/cs-batch-test:amd64
# amd64: digest:
sha256:99c40947b3ed4352d705c1c6b5e87f168223be980ed13112e7f6a601c5ff7587
```

Confirm in ECR:

```
aws ecr describe-images --repository-name cs-batch-test --region us-east-1 \
  --query 'imageDetails[0].
{tag:imageTags[0],digest:imageDigest,arch:imageManifestMediaType}'
```

Pull and Push the Falcon Sensor Image

Pull via CrowdStrike pull script

```
export FALCON_CLIENT_ID=<your_client_id>
export FALCON_CLIENT_SECRET=<your_client_secret>

LATESTSENSOR=$(bash <(curl -Ls \
  https://github.com/CrowdStrike/falcon-scripts/releases/latest/download/falcon-
container-sensor-pull.sh) \
  -u $FALCON_CLIENT_ID -s $FALCON_CLIENT_SECRET \
  -t falcon-container --platform x86_64 2>&1 | tail -1)

echo $LATESTSENSOR
# registry.crowdstrike.com/falcon-container/release/falcon-container:7.39.0-7802
```

Important: The --platform x86_64 flag in the pull script controls which image architecture is pulled. This validated run used 7.39.0-7802.

Force-pull as linux/amd64 on Apple Silicon

```
# On Apple Silicon, Docker may pull arm64 by default. Force amd64:
docker pull --platform linux/amd64 "$LATESTSENSOR"
# 7.39.0-7802: Pulling from falcon-container/release/falcon-container
# Digest: sha256:518b8782f29df7beceace5a2f930049495950c1873fe246ee5b7595cc93a3f60
```

Create ECR Repo and Push

```
aws ecr create-repository \
  --repository-name falcon-sensor/falcon-container \
  --region us-east-1

docker tag "$LATESTSENSOR" \
  597047870845.dkr.ecr.us-east-1.amazonaws.com/falcon-sensor/falcon-
container:amd64

docker push \
  597047870845.dkr.ecr.us-east-1.amazonaws.com/falcon-sensor/falcon-
container:amd64
# amd64: digest:
sha256:669e171f5704436c3da274799f75d8e4bf3dbdebc7094caeb7968ea664e49e74
```

Patch the Application Image with falconutil

Problem: macOS Keychain blocks falconutil

When you run falconutil as a Docker container and mount ~/.docker/config.json, the config on macOS has:

```
{
  "credsStore": "osxkeychain"
}
```

The falconutil Linux container cannot call docker-credential-osxkeychain because that binary doesn't exist inside it. Result:

```
Error: failed to pull/inspect source image: failed to pull image '...'
(credentials tried: 2)
```

Fix: Generate a fresh ECR token and write it as a raw Base64 credential to a temp config file:

```
ECR_TOKEN=$(aws ecr get-login-password --region us-east-1)
ECR_AUTH=$(echo -n "AWS:${ECR_TOKEN}" | base64)
```

```
mkdir -p /tmp/docker-ecr-config
cat > /tmp/docker-ecr-config/config.json <<EOF
{
  "auths": {
    "597047870845.dkr.ecr.us-east-1.amazonaws.com": {
      "auth": "${ECR_AUTH}"
    }
  }
}
EOF
```

Problem: Platform mismatch on Apple Silicon

Running the falconutil container without specifying --platform linux/amd64 causes:

```
WARNING: The requested image's platform (linux/amd64) does not match the detected
host platform (linux/arm64/v8)
```

```
...
```

```
Error: failed to pull falcon image: failed to pull image '...' for platform
'linux/arm64/v8' (credentials tried: 1)
```

falconutil auto-detects the source image platform and then tries to pull the Falcon image for that same platform. Since the source image was built as arm64, it tries to pull the Falcon sensor as arm64. The fix is explicit --platform linux/amd64 on both the docker run command **and** the falconutil flag.

Working falconutil command

```
aws ecr create-repository --repository-name cs-batch-test-patched --region us-east-1
```

```
docker run --user 0:0 \
```

```

-v /tmp/docker-ecr-config/config.json:/root/.docker/config.json \
-v /var/run/docker.sock:/var/run/docker.sock \
--platform linux/amd64 \
--rm 597047870845.dkr.ecr.us-east-1.amazonaws.com/falcon-sensor/falcon-
container:amd64 \
    falconutil patch-image \
--source-image-uri 597047870845.dkr.ecr.us-east-1.amazonaws.com/cs-batch-
test:amd64 \
--target-image-uri 597047870845.dkr.ecr.us-east-1.amazonaws.com/cs-batch-test-
patched:latest \
--falcon-image-uri 597047870845.dkr.ecr.us-east-1.amazonaws.com/falcon-sensor/
falcon-container:amd64 \
--cid 4344FD98B7B242519A94CD52E0345CD9-52 \
--cloud-service ECS_FARGATE \
--container cs-batch-test \
--platform linux/amd64 \
--image-pull-policy IfNotPresent

```

Successful output:

```

time="..." level=info msg="Using user-specified platform for both images: linux/
amd64"
time="..." level=info msg="Pulling source image for platform linux/amd64"
time="..." level=info msg="Pulling falcon image for platform linux/amd64"
time="..." level=info msg="Requested platform validation passed: linux/amd64"
time="..." level=info msg="Platform compatibility check passed: linux/amd64"
⇒ [internal] load remote build context
⇒ [stage-1 1/2] FROM .../cs-batch-test:amd64
⇒ [build 1/7] FROM .../falcon-sensor/falcon-container:amd64
⇒ [build 2/7] RUN mkdir -p /tmp/CrowdStrike/rootfs/usr/bin && cp -R /usr/bin/
falcon* ...
⇒ [build 3/7] RUN cp -R /usr/lib64 /tmp/CrowdStrike/rootfs/usr/
⇒ [build 4/7] RUN mkdir -p /tmp/CrowdStrike/rootfs/usr/lib && cp -R /usr/lib/
locale ...
⇒ [build 5/7] RUN cd /tmp/CrowdStrike/rootfs && ln -s usr/bin bin && ln -s usr/
lib64 lib64 ...
⇒ [build 6/7] RUN mkdir -p /tmp/CrowdStrike/rootfs/etc/ssl/certs && cp /etc/ssl/
certs/ca-bundle* ...
⇒ [build 7/7] RUN chmod -R a=rX /tmp/CrowdStrike
⇒ [stage-1 2/2] COPY --from=build /tmp/CrowdStrike /opt/CrowdStrike
⇒ exporting layers
⇒ writing image
sha256:744e5d14ede32a2a63368073b077b2c030e9a8db1b7fe0186db2131cec336411
⇒ naming to .../cs-batch-test-patched:latest
⇒ Successfully built image ID:
sha256:744e5d14ede32a2a63368073b077b2c030e9a8db1b7fe0186db2131cec336411

```

Push the patched image

```

docker push 597047870845.dkr.ecr.us-east-1.amazonaws.com/cs-batch-test-
patched:latest

```

```
# latest: digest:
sha256:e098114ae3be1ef328fc6c70da755e549d1cc67550e9dc844e85e92a1ee9431b size:
1571
```

Inspect the Patched Image

This is a critical verification step. Run this after `falconutil` to confirm the image was correctly modified before deploying:

```
docker inspect 597047870845.dkr.ecr.us-east-1.amazonaws.com/cs-batch-test-
patched:latest \
  --format '{{json .Config}}' | python3 -c "
import json, sys
cfg = json.load(sys.stdin)
print('Entrypoint:', cfg.get('Entrypoint'))
print('Cmd:', cfg.get('Cmd'))
print('Env (CrowdStrike):', [e for e in (cfg.get('Env') or []) if 'CS_' in e or
'FALCON' in e])
"
```

Expected output — confirm all three things:

```
Entrypoint: ['/opt/CrowdStrike/rootfs/bin/falcon-entrypoint', '/app/
entrypoint.sh']
Cmd: None
Env (CrowdStrike): [
  'FALCONCTL_OPTS=--cid=4344FD98B7B242519A94CD52E0345CD9-52',
  'CS_CONTAINER=cs-batch-test',
  'CS_CLOUD_SERVICE=ECS_FARGATE',
  '__CS_FALCON_SENSOR_ROOT=/opt/CrowdStrike/rootfs'
]
```

Field			What it means
Entrypoint	starts with	/opt/CrowdStrike/...	Falcon sensor wraps the original entrypoint
FALCONCTL_OPTS=	--cid=...		CID correctly embedded — sensor will use this on startup
CS_CLOUD_SERVICE=ECS_FARGATE			Tells sensor it's running on Fargate (userspace mode, no ptrace needed)
__CS_FALCON_SENSOR_ROOT			Sensor binaries are at /opt/CrowdStrike/rootfs inside the container

If the CID is wrong or missing in `FALCONCTL_OPTS`, the sensor will start but fail to register. Re-run `falconutil` with the correct `--cid`.

If `CS_CLOUD_SERVICE` is missing, the sensor may attempt kernel-mode injection, which fails on Fargate. Always pass `--cloud-service ECS_FARGATE`.

AWS Batch Infrastructure Setup

IAM Role

```
# Verify role exists
aws iam get-role --role-name ecsTaskExecutionRole --query 'Role.Arn' --output
text
# arn:aws:iam::597047870845:role/ecsTaskExecutionRole

# Verify required policies
aws iam list-attached-role-policies --role-name ecsTaskExecutionRole \
  --query 'AttachedPolicies[].PolicyName' --output text
# AmazonECSTaskExecutionRolePolicy    CloudWatchLogsFullAccess
```

AmazonECSTaskExecutionRolePolicy grants ECR image pull and CloudWatch Logs write. Without it, the Fargate task will fail with CannotPullContainerError.

Compute Environment

```
aws batch create-compute-environment \
  --compute-environment-name cs-fargate-ce \
  --type MANAGED \
  --state ENABLED \
  --compute-resources '{
    "type": "FARGATE",
    "maxvCpus": 256,
    "subnets": [
      "subnet-059a781a5ded5d9cb",
      "subnet-0013bb1850fa8fedf",
      "subnet-03f3531990ec51f16",
      "subnet-0bd7314cc4a541e24",
      "subnet-09e75cdd99bfb4d51",
      "subnet-09ba2e8dd6e555d3e"
    ],
    "securityGroupIds": ["sg-0e19a5a2b0ebeda19"]
  }' \
  --region us-east-1
```

Verified state:

```
{
  "computeEnvironmentName": "cs-fargate-ce",
  "computeEnvironmentArn": "arn:aws:batch:us-east-1:597047870845:compute-
environment/cs-fargate-ce",
  "ecsClusterArn": "arn:aws:ecs:us-east-1:597047870845:cluster/AWSBatch-cs-
fargate-ce-691a9cdb-b2c8-3642-9a5d-c5825e04119b",
  "type": "MANAGED",
  "state": "ENABLED",
  "status": "VALID",
  "statusReason": "ComputeEnvironment Healthy",
}
```



```
    "containerOrchestrationType": "ECS"
}
```

Poll until status = VALID before creating the job queue:

```
# Poll (Batch does not have a native wait command for this)
until [ "$(aws batch describe-compute-environments \
    --compute-environments cs-fargate-ce \
    --query 'computeEnvironments[0].status' --output text --region us-east-1)" =
"VALID" ]; do
    sleep 5
done
```

Job Queue

```
aws batch create-job-queue \
    --job-queue-name cs-fargate-queue \
    --state ENABLED \
    --priority 100 \
    --compute-environment-order '[{"order": 1, "computeEnvironment": "cs-fargate-
ce"}]' \
    --region us-east-1
```

Poll until VALID:

```
until [ "$(aws batch describe-job-queues \
    --job-queues cs-fargate-queue \
    --query 'jobQueues[0].status' --output text --region us-east-1)" = "VALID" ];
do
    sleep 5
done
```

Job Definition Details

Important: SYS_PTRACE and AWS Batch

The CrowdStrike documentation says to add SYS_PTRACE to linuxParameters.capabilities. **This field does not exist in the AWS Batch API** — neither in containerProperties.linuxParameters nor in ecsProperties.taskProperties[].containers[].linuxParameters. Attempting to set it produces:

```
Parameter validation failed:
Unknown parameter in containerProperties.linuxParameters: "capabilities",
must be one of: devices, initProcessEnabled, sharedMemorySize, tmpfs, maxSwap,
swappiness
```

Why it still works: The CrowdStrike docs requirement for SYS_PTRACE applies to the **task definition patching** approach, where the Falcon init container injects into the app process via ptrace. With falconutil image patching and CS_CLOUD_SERVICE=ECS_FARGATE, the sensor runs

in **userspace injection mode** — it wraps the entrypoint directly and does not use ptrace at all. No capability escalation is needed.

Register the Job Definition

```
aws logs create-log-group --log-group-name /aws/batch/cs-falcon-job --region us-east-1
```

```
aws batch register-job-definition \
  --job-definition-name cs-falcon-job \
  --type container \
  --platform-capabilities ['FARGATE'] \
  --container-properties '{
    "image": "597047870845.dkr.ecr.us-east-1.amazonaws.com/cs-batch-test-patched:latest",
    "resourceRequirements": [
      {"type": "VCPU", "value": "0.25"},
      {"type": "MEMORY", "value": "512"}
    ],
    "jobRoleArn": "arn:aws:iam::597047870845:role/ecsTaskExecutionRole",
    "executionRoleArn": "arn:aws:iam::597047870845:role/ecsTaskExecutionRole",
    "networkConfiguration": {
      "assignPublicIp": "ENABLED"
    },
    "fargatePlatformConfiguration": {
      "platformVersion": "LATEST"
    },
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/aws/batch/cs-falcon-job",
        "awslogs-region": "us-east-1",
        "awslogs-stream-prefix": "batch"
      }
    }
  }' \
  --region us-east-1
```

Registered as:

```
{
  "jobDefinitionName": "cs-falcon-job",
  "jobDefinitionArn": "arn:aws:batch:us-east-1:597047870845:job-definition/cs-falcon-job:1",
  "revision": 1,
  "status": "ACTIVE",
  "type": "container",
  "platformCapabilities": ["FARGATE"],
  "containerOrchestrationType": "ECS"
}
```

Note on networkConfiguration.assignPublicIp: ENABLED: This is required if your subnets are public (default VPC). If your subnets are private with a NAT gateway, set this to DISABLED. The sensor must be able to reach the Falcon cloud (ts01-b.cloudsink.net on port 443) to register. Without outbound internet access, the sensor will run but never register in the console.

Submit and Monitor the Batch Job

Submit

```
JOB_ID=$(aws batch submit-job \
  --job-name cs-falcon-test-run \
  --job-queue cs-fargate-queue \
  --job-definition cs-falcon-job \
  --region us-east-1 \
  --query 'jobId' --output text)

echo "Job ID: $JOB_ID"
# 12fd13a4-09e0-46b7-bb97-63c5ea859593
```

Monitor Job Status

AWS Batch job states in order: SUBMITTED → PENDING → RUNNABLE → STARTING → RUNNING → SUCCEEDED or FAILED

```
aws batch describe-jobs --jobs $JOB_ID --region us-east-1 \
  --query 'jobs[0].{status:status,reason:statusReason,taskArn:container.taskArn}'
```

Actual progression observed:

```
[10:15:06] STARTING
[10:15:22] STARTING
[10:15:38] STARTING
[10:15:53] RUNNING
```

Time from submission to RUNNING: ~50 seconds (Fargate cold start).

Get the ECS Task ARN (needed for verifying in ECS and Falcon console):

```
aws batch describe-jobs --jobs $JOB_ID --region us-east-1 \
  --query 'jobs[0].container.taskArn' --output text
# arn:aws:ecs:us-east-1:597047870845:task/AWSBatch-cs-fargate-
ce-691a9cdb-.../2bfa594896aa4323a7e186fcf7517ea0
```

The short task ID is the last segment: 2bfa594896aa4323a7e186fcf7517ea0. This is the **Pod ID** used in the Falcon console.

CloudWatch Log Verification

CloudWatch logs are the primary way to confirm the Falcon sensor wrapped and launched the application correctly.

Log stream naming

AWS Batch + awslogs driver names streams as:

```
{prefix}/{container_name}/{task_id}
```

In this deployment:

```
batch/default/2bfa594896aa4323a7e186fcf7517ea0
```

Fetch logs

```
aws logs get-log-events \
  --log-group-name /aws/batch/cs-falcon-job \
  --log-stream-name "batch/default/2bfa594896aa4323a7e186fcf7517ea0" \
  --region us-east-1 \
  --query 'events[].message' \
  --output text
```

Actual log output from this deployment:

```
=== CS Batch Test Job Starting ===
Hostname: ip-172-31-53-231.ec2.internal
Date: Thu Jul  9 15:15:51 UTC 2026
[15:15:51] Iteration 1 – workload running
[15:16:21] Iteration 2 – workload running
[15:16:51] Iteration 3 – workload running
[15:17:21] Iteration 4 – workload running
[15:17:51] Iteration 5 – workload running
[15:18:21] Iteration 6 – workload running
[15:18:51] Iteration 7 – workload running
[15:19:21] Iteration 8 – workload running
[15:19:51] Iteration 9 – workload running
[15:20:21] Iteration 10 – workload running
```

What this proves: - The Falcon entrypoint ran (/opt/CrowdStrike/rootfs/bin/falcon-entrypoint) without error - It correctly handed off to the application's entrypoint.sh - The app has been running cleanly for 5+ minutes with consistent 30-second iteration intervals

What you will NOT see in these logs: The Falcon sensor's own startup messages. The sensor starts before the application entrypoint runs, but its output goes to stderr inside the container process and is not forwarded to CloudWatch unless specifically configured. The absence of sensor logs here is **normal** — the sensor either registered successfully (silent) or failed (check Falcon console).

Listing all log streams in the log group

```
aws logs describe-log-streams \
  --log-group-name /aws/batch/cs-falcon-job \
  --region us-east-1 \
  --query 'logStreams[].{stream:logStreamName,last:lastEventTimestamp}'
```

ECS Task Verification

Because AWS Batch on Fargate creates real ECS tasks, you can query them directly via the ECS API.

```
aws ecs describe-tasks \
  --cluster AWSBatch-cs-fargate-ce-691a9cdb-b2c8-3642-9a5d-c5825e04119b \
  --tasks 2bfa594896aa4323a7e186fcf7517ea0 \
  --region us-east-1
```

Key fields from the actual task:

Field	Value	Significance
lastStatus	RUNNING	Task is alive
launchType	FARGATE	Confirmed Fargate (not EC2)
platformVersion	1.4.0	Fargate platform — 1.4.0 is required for awsvpc networking
ecs.cpu-architecture	x86_64	Confirms correct amd64 platform
connectivity	CONNECTED	Network interface is up
connectivityAt	2026-07-09T10:15:16	ENI was attached
pullStartedAt	2026-07-09T10:15:41	ECR image pull began
pullStoppedAt	2026-07-09T10:15:49	ECR image pull complete (8 seconds)
startedAt	2026-07-09T10:15:49	Container started
privateIPv4Address	172.31.53.231	Task IP in VPC
subnetId	subnet-059a781a5ded5d9cb	us-east-1e subnet
networkInterfaceId	eni-09470c13398a59a36	ENI attached to the task
enableExecuteCommand	false	ECS Exec disabled (Batch default)

Getting the ECS cluster name from Batch:

```
aws batch describe-compute-environments \
  --compute-environments cs-fargate-ce \
  --query 'computeEnvironments[0].ecsClusterArn' --output text --region us-east-1
# arn:aws:ecs:us-east-1:597047870845:cluster/AWSBatch-cs-fargate-ce-691a9cdb-b2c8-3642-9a5d-c5825e04119b
```

The cluster name is everything after cluster/.

Note on ECS Exec

ECS Exec (`aws ecs execute-command`) is **not enabled** by default for Batch-managed tasks. Enabling it requires `enableExecuteCommand: true` in the task definition, which Batch does not expose as a parameter. If you need to exec into the container to run `falconctl -g --aid`, you have two options:

1. Run the job definition directly as an ECS task (not via Batch) with `enableExecuteCommand: true`
 2. Use Batch with a custom container that writes the AID to a file/log on startup
-

Falcon Console Verification

Validated 2026-07-09: Host appeared in the Falcon console with a valid AID within ~2 minutes of the Batch job reaching RUNNING state.

Navigate to Host Management

1. Log into the Falcon console at your cloud URL (this deployment uses **us-2**)
2. Go to: **Host setup and management** → **Manage endpoints** → **Host management**
3. Add a filter: **Pod ID** = `2bfa594896aa4323a7e186fcf7517ea0`

The Pod ID in Falcon corresponds directly to the ECS Task ID from AWS.

What a successful registration looks like

- The host appears within 1–2 minutes of the container entering RUNNING state
- **AID (Agent ID):** A 32-character hex string — this is the unique sensor instance identifier
- **Hostname:** Will show the Fargate task's internal hostname (matches `ip-172-31-53-231.ec2.internal` from the logs)
- **OS:** Linux / Amazon Linux (Fargate host)
- **Sensor version:** Should match `7.39.0-7802`
- **Last seen:** Updates every few minutes while the container is running

Validate sensor is alive from within the container

If ECS Exec is enabled, run:

```
aws ecs execute-command \
  --cluster AWSBatch-cs-fargate-ce-691a9cdb-b2c8-3642-9a5d-c5825e04119b \
  --task 2bfa594896aa4323a7e186fcf7517ea0 \
  --container default \
  --interactive \
  --command "/tmp/CrowdStrike/rootfs/bin/falconctl -g --aid"
```

A registered sensor will output a 32-character hex AID. An empty AID means the sensor started but hasn't connected yet (check outbound connectivity to CrowdStrike cloud).

Troubleshooting Reference

Job stuck in RUNNABLE

The job can't be placed on a compute environment. Check:

```
# Is the compute environment VALID and ENABLED?
aws batch describe-compute-environments --compute-environments cs-fargate-ce \
  --query 'computeEnvironments[0].
{status:status,state:state,reason:statusReason}' \
  --region us-east-1

# Is the job queue VALID and pointing to the right CE?
aws batch describe-job-queues --job-queues cs-fargate-queue \
  --query 'jobQueues[0].{status:status,state:state,ces:computeEnvironmentOrder}' \
  --region us-east-1
```

Job fails with CannotPullContainerError

The ECS task can't pull the image from ECR. Common causes: - executionRoleArn is missing from the job definition (required for Fargate ECR pulls) - The role doesn't have AmazonECSTaskExecutionRolePolicy attached - The ECR repo is in a different region than the Fargate task - assignPublicIp: ENABLED is missing and the subnet has no NAT gateway

```
# Check the task's stopped reason
aws ecs describe-tasks \
  --cluster <cluster-name> --tasks <task-id> --region us-east-1 \
  --query 'tasks[0].stoppedReason'
```

Job reaches RUNNING but Falcon console shows no host

The sensor is starting but can't phone home. Outbound networking issue: - Fargate task is in a private subnet without a NAT gateway → set assignPublicIp: ENABLED or add NAT - Security group blocks outbound 443 → add outbound rule for 0.0.0.0/0 port 443 - FALCONCTL_OPTS has the wrong CID → re-inspect the patched image (see Inspect the Patched Image) - Wrong Falcon cloud (e.g., credentials are us-2 but sensor was configured for us-1) → verify your Falcon console URL

falconutil fails with “credentials tried: N”

ECR auth issue from inside the falconutil container: - The config.json uses "credsStore": "osxkeychain" → use the raw-token workaround from Phase 4 - The ECR token has expired (tokens last 12 hours) → regenerate with `aws ecr get-login-password` - The ECR repo is in a different AWS account → include cross-account ECR credentials in the config.json

falconutil fails with platform mismatch errors

Running on Apple Silicon and seeing arm64 errors: - Add `--platform linux/amd64` to both the `docker run` command and as a falconutil flag - Confirm source and Falcon images are both in local cache as amd64: `docker images --format '{{.Repository}}:{{.Tag}} {{.ID}}'`

Image entrypoint not modified after falconutil runs

Inspect the patched image immediately after the build:

```
docker inspect <patched-image-uri> --format '{{json .Config.Entrypoint}}'
```

If this returns the original entrypoint (e.g., ["/app/entrypoint.sh"]) instead of starting with /opt/CrowdStrike/..., the patch did not apply. Re-run falconutil and confirm you see all 7 build steps in the output.

AWS Batch containerProperties.linuxParameters “capabilities” error

Unknown parameter in containerProperties.linuxParameters: "capabilities"

This is expected. AWS Batch does not expose Linux capabilities through its API. Use the falconutil patching approach with --cloud-service ECS_FARGATE instead of task definition patching. The ECS_FARGATE cloud service mode does not require SYS_PTRACE.

Summary of Verified Resources

Resource	Name / ARN	Status
ECR repo (app)	cs-batch-test	✓
ECR repo (sensor)	falcon-sensor/falcon-container	✓
ECR repo (patched)	cs-batch-test-patched	✓
ECR image (app)	1561c098114e3be1ef328fc6c70da755e549d1cc67550e9dc844e85e93a1be9431b	✓ pushed
IAM role	ecsTaskExecutionRole	✓ has required policies
Batch compute env	cs-fargate-ce	✓ VALID / ENABLED
Batch job queue	cs-fargate-queue	✓ VALID / ENABLED
Batch job definition	cs-falcon-job:1	✓ ACTIVE
CloudWatch log group	/aws/batch/cs-falcon-job	✓ receiving logs
Batch job	12fd13a4-09e0-46b7-bb97-63c5ea859593	✓ RUNNING
ECS task	2bfa594896aa4323a7e186fcf7517ea0	✓ RUNNING
Sensor version	7.39.0-7802	✓
Platform	x86_64 / Fargate 1.4.0	✓